



**SCHOOL OF MATHEMATICS AND APPLIED SCIENCES**  
**DEPARTMENT OF COMPUTING**

TO : MR. S. YUTE

FROM : SIMON MHONE & TIYAMIKE NGOMA

REGISTRATION NO : BSC/29/23 & BSC/COM/NE/24/23

COURSE NAME : NETWORK PROGRAMMING AND APPLICATION  
DEVELOPMENT

COURSE CODE : NET322

ASSIGNMENT TITLE : *Serialization, Web Services, Concurrency & HTTP Mechanics*

DUE DATE : 8 , 2026, MAY

# Assignment 1 — Theory Paper

## A. Serialization Choices

---

**YAML** is used for configuration of files. Field technicians must adjust sampling intervals and server addresses without specialist tooling; YAML's readable, comment-friendly syntax serves this directly. Verbosity is irrelevant because config is parsed once at startup, not on a hot path. Schema validation via *yamale* catches misconfigurations before deployment across dispersed greenhouse nodes.

**Protobuf** governs the sensor-to-server link. Four sensor types per greenhouse transmit at regular intervals over Malawi's bandwidth-constrained mobile links. Protobuf's binary encoding is 3–10× smaller than JSON and parses with negligible CPU overhead — critical for low-cost microcontrollers and metered data. Typed *.proto* schemas enforce field contracts and surface version mismatches at compile time.

**JSON and XML** serve the public REST API via content negotiation. The mobile farmer app requests *Accept: application/json* for its concise, natively-parsed format; the legacy desktop analytics tool sends *Accept: application/xml* to feed its fixed XSLT pipeline. Both clients are served from one endpoint — keeping the API surface minimal and avoiding a costly toolchain replacement.

## B. Web Services Protocol

---

**REST** fits the historical-data API because the cooperative's data is naturally resource-oriented: greenhouses, sensors, and time-series readings map cleanly onto URLs (e.g., */greenhouses/{id}/sensors/{s}/readings?from=...*). Statelessness means the server can be load-balanced without session affinity — and a mobile app can retry a failed request without state coordination, essential over unreliable rural connections. *Cache-Control* headers reduce repeated data transfer costs on metered links.

**SOAP** adds XML envelope overhead per call and implies statefulness, conflicting with low-bandwidth goals and horizontal scaling. **RPC protocols** such as gRPC are action-oriented and require HTTP/2 binary framing, which the legacy XML desktop client cannot consume without a full re-engineering effort. REST's uniform interface and universal library support make it the pragmatic, low-maintenance choice.

## C. AsyncIO vs Threading

---

**The server is I/O-bound:** at any moment it waits on sensor TCP frames, database writes, or WebSocket flushes — never CPU computation. Python's GIL means threads give no concurrency benefit for I/O; a thread-per-connection design for 20 greenhouses × 4 sensors

wastes ~40 MB of stack memory on blocked waits, with kernel-scheduler overhead on every context switch.

**asyncio** coroutines on a single event loop are the right model. Each coroutine yields at *await* points, returning control so other ready tasks proceed. Thousands of connections share one OS thread with no per-connection stack; coroutine switches are pure-Python and far cheaper than kernel thread switches — reducing hardware requirements on the cooperative’s budget server.

**Threads** remain appropriate via *run\_in\_executor()* for blocking C library calls (e.g., serial-port drivers with no async API). **Processes** (ProcessPoolExecutor) suit future CPU-bound workloads such as ML-based anomaly detection, where true multi-core parallelism is needed beyond the GIL.

## D. HTTP Mechanics

---

**Content negotiation** directly solves the multi-client requirement. The server inspects the *Accept* header and serializes the same data as JSON or XML accordingly, eliminating separate endpoints and keeping API maintenance simple — a meaningful saving for a small cooperative with limited developer resources.

**The WebSocket upgrade handshake** enables the live dashboard at minimal cost. The farmer’s browser sends *GET /live Upgrade: websocket*; the server replies *101 Switching Protocols*, opening a full-duplex channel. Every persisted reading is then pushed to all connected dashboards instantly — no polling, no repeated TLS handshakes. Crucially, the upgrade reuses port 443 and traverses NAT and ISP-level firewalls transparently, which matters on Malawian mobile networks where non-standard ports are often blocked.

---

## References

- [1] Google. (2023). *Protocol Buffers Developer Guide*. <https://protobuf.dev/overview/>
- [2] Fielding, R. T. (2000). *Architectural Styles and the Design of Network-based Software Architectures*. UC Irvine.
- [3] Python Software Foundation. (2024). *asyncio — Asynchronous I/O*. <https://docs.python.org/3/library/asyncio.html>
- [4] Fette, I., & Melnikov, A. (2011). *The WebSocket Protocol*. RFC 6455. IETF.
- [5] Fielding, R., et al. (2022). *HTTP Semantics*. RFC 9110. IETF.